

Das System ist speziell für Events gedacht, die keine feste Struktur haben und bei denen die Organisationsform spontan ist. Dies wirkt sich direkt auf die Bedienung und die Kontrolle der angezeigten Inhalte aus. In solchen Situationen erfolgt die Steuerung meist durch ein ad-hoc Krisenmanagement. Daher muss die Bedienung einfach, selbsterklärend und ortsunabhängig möglich sein – möglichst ohne vorherige Schulung oder Einweisung.

Die Steuerungs- und Bedienteile des Kiosk-Systems sind passwortgeschützt, um unbefugte Eingriffe zu verhindern.

Das Kiosk-System zeigt auf einem Präsentationsbildschirm Inhalte wie Videos, Bilder oder Slideshows an. Diese Inhalte können zentral über eine Weboberfläche gesteuert werden. Änderungen wirken sofort, sodass Durchsagen, Notfallmeldungen oder Sponsor-Clips unmittelbar auf den Bildschirmen erscheinen.

Das Kiosk-Programm besteht aus mehreren Programmteilen und ist modular aufgebaut:

- **Server** als zentrale Steuerung.
- **Webseiten** (Präsentation, Remote, Admin) als Clients.
- **StreamDeck-Integration** als optionales Bediengerät.

Server (app.py)

- Herzstück des Systems, geschrieben in Python mit dem Web-Framework Flask und Flask-SocketIO.
- Stellt die Weboberflächen (Admin, Remote, Präsentation) bereit.
- Verwaltet den aktuellen Medienstatus (z. B. welches Video gerade läuft).
- Kommuniziert über **WebSockets** mit den Clients, sodass Änderungen sofort sichtbar werden.
- Enthält eine Scheduler-Funktion, die zu einer festgelegten Uhrzeit automatisch ein Standardmedium startet (z. B. Sponsorclip nachts um 02:00)

Präsentationsoberfläche (presentation.html)

- Läuft im Browser (z. B. Vollbildmodus auf einem zweiten Monitor oder Beamer).
- Empfängt über **SocketIO** Anweisungen vom Server und zeigt Bilder, Videos, Slideshows oder Playlists.
- Kümmt sich um Übergänge (Start/Stop von Slideshows oder Videos) und blendet beim Start kurz die IP-Adresse ein, damit man den Rechner leicht identifizieren kann.

Remote-Oberfläche (remote.html)

- Ebenfalls browserbasiert, für Bediener gedacht.
- Bietet eine einfache Oberfläche mit Buttons, die bestimmte Inhalte starten.
- Zeigt eine kleine Vorschau (z. B. das aktuelle Bild einer Slideshow oder ein Video-Thumbnail).
- Holt sich die Button-Belegung aus einer zentralen Konfigurationsdatei (button_config.json)

Admin-Oberfläche (admin.html)

- Dient zur technischen Übersicht.
- Zeigt an, ob Präsentations- oder Remote-Clients verbunden sind, welcher Inhalt aktiv ist und ob der Server erreichbar ist.
- Heartbeat um die Serververbindung zu prüfen.
- Hier lassen sich auch Button-Belegungen ändern oder das StreamDeck ein- und ausschalten.

StreamDeck-Integration (app_streamdeck.py)

- Ein separates Python-Programm, das ein Elgato StreamDeck (egal welches Modell) einbindet.
- Liest die button_config.json und zeigt die konfigurierten Inhalte als Tasten auf dem StreamDeck an.
- Bei Tastendruck sendet es API-Befehle an den Server, um Medien zu wechseln.
- Der Status der Tasten (z. B. aktive Markierung) wird über **SocketIO** vom Server zurückgemeldet.
- Defaultmässig ist die Streamdeck Bedienung gesperrt und dient nur als Anzeige

Die Kommunikation erfolgt auf zwei Wegen:

Die Kommunikation läuft überwiegend über SocketIO (für Echtzeit-Synchronisation) und ergänzend über eine einfache API. Dank dieser Struktur ist das System flexibel, leicht erweiterbar und kann sowohl in kleinen Umgebungen (ein PC, ein Bildschirm) als auch in größeren Setups (mehrere Monitore, mehrere Bediener) eingesetzt werden.

HTTP-API (REST-ähnlich):

- Der Server stellt eine API unter /api?Function=... bereit.
- Damit können externe Programme (z. B. das StreamDeck) Kommandos wie video1, bild3 oder playlist senden.

WebSockets (SocketIO):

- Sorgen für Echtzeitkommunikation.
- Beispiel: Wenn ein Remote-Client „Video 2“ startet, wird ein show_media-Event an alle verbundenen Präsentations- und Remote-Clients geschickt. Diese wechseln sofort auf den neuen Inhalt.
- Weitere Events: slideshow_image (aktuelles Bild der Slideshow), heartbeat_request/response (Verbindungstest), reload_config (Buttonbelegung aktualisieren)

Programmiersprachen und Erweiterungen

Python 3 als Hauptsprache für die Serverlogik.

- Bibliotheken/Frameworks:
 - **Flask** (Webserver)
 - **Flask-SocketIO** (Echtzeit-Kommunikation)
 - **psutil** (Systeminfos wie CPU/RAM-Auslastung im Terminal)
 - **Pillow** (Bildbearbeitung fürs StreamDeck)
 - **StreamDeck SDK** (Steuerung der Elgato-Hardware)
- **HTML, CSS und JavaScript** für die Client-Oberflächen (Admin, Remote, Präsentation).
- **socket.io.js** als JavaScript-Bibliothek zur WebSocket-Kommunikation im Browser.

SDI Ausgabe

Falls eine SDI-Ausgabe benötigt wird, kann das HDMI-Signal des Präsentationsrechners mit einem HDMI-zu-SDI Wandler (Converter) umgesetzt werden. Auf diese Weise lässt sich das Signal in bestehende Videotechnik oder Regien einspeisen. Über die Windows-Bildschirmeinstellungen kann die Ausgabe so konfiguriert werden, dass ein Signal im Format 1080i50 (Full-HD, interlaced, 50 Hz) bereitgestellt wird.

HDMI-zu-SDI-Ausgabe unter Windows 11

Monitor anschließen

- Den Präsentationsrechner per HDMI an den HDMI→SDI-Wandler anschließen.
- Der Wandler ist dann mit der SDI-Infrastruktur verbunden (z. B. Videomischer, Kreuzschiene).

Anzeigeeinstellungen öffnen

- Rechtsklick auf den Desktop → Anzeigeeinstellungen.
- Dort den angeschlossenen Bildschirm (HDMI-Ausgang) auswählen.

Auflösung einstellen

- Unter Anzeigeauflösung z.B. 1920 × 1080 auswählen.
- Falls Windows automatisch 60 Hz vorschlägt, den nächsten Schritt beachten.

Erweiterte Anzeigeeinstellungen

- Unten auf Erweiterte Anzeigeeinstellungen klicken.
- Bei den Anzeigeeinstellungen Adaptereigenschaften für Bildschirm x anzeigen
- Alle Modi auflisten
- Auflösung und Bildwiederholfrequenz 50 Hz interlaced einstellen.

Je nach Grafikkartentreiber (Intel, AMD, NVIDIA) erscheint bei den unterstützten Modi 1080i, 50 Hz. Falls nur 1080p50 angeboten wird, muss auf einen Cross Converter zurückgegriffen werden.

Kiosk-System – Technische Anpassungen & Administration

Diverse Einstellungen können nur im Code geändert werden. Dies kann mit einem beliebigen Text Editor erfolgen. Mache zur Sicherheit vorher eine Kopie von App.py und app_streamdeck.py

1) Server-Port, Zugangsdaten & Schutz (Python-Server)

Datei: app.py

- Port ändern: Variable 'port = 53100'. Muss mit allen Clients übereinstimmen.
- Passwortschutz für Bedienoberflächen:
password_protection = True
USERNAME = "admin"
PASSWORD = "password123" → hier anpassen.
- API-Schutz (optional): API_protection = True setzen, wenn auch die API abgesichert sein soll.
- Standard-Inhalt & Zeitplan: DEFAULT_PROGRAM und SCHEDULE_PROGRAM definieren, sowie SCHEDULE_TIME (z. B. '02:00').

2) StreamDeck-Treiber anpassen

Datei: app_streamdeck.py

- API_BASE_URL: Hier muss derselbe Port eingetragen werden wie in app.py.
 - STREAMDECK_INPUT_LOCK: True = Tasten beim Start gesperrt.
- Hinweis: Portänderungen immer synchron in Server und StreamDeck-Code durchführen.

3) Präsentations- und Bedien-Webseiten

Datei: presentation.html

- slideshowTimer: Intervall für Bildwechsel in ms (3000 = 3s).
- IP_BADGE_MINUTES: Dauer der IP-Einblendung in Minuten.
- Standardstart: showVideo('videos/video1.mp4') oder showSlideshow('bild1/').

Datei: admin.html

- Admin-Panel zeigt Status, Heartbeat und erlaubt das Freigeben/Sperren des StreamDeck.

Datei: remote.html

- Buttons werden dynamisch aus button_config.json geladen und können über das Admin-Panel angepasst werden.

4) Button-Belegung

Datei: button_config.json

- Struktur: enabled, label, type (slideshow/video/playlist), src (Pfad).
- Bearbeitung: Über das Admin-Panel → Button-Konfiguration anpassen und speichern.
- Änderungen werden an alle verbundenen Clients verteilt (reload_clients).

5) Sicherheit

- Standard-Benutzername und Passwort müssen vor Einsatz geändert werden.
- API-Schutz aktivieren, wenn externe Systeme beteiligt sind.
- StreamDeck-Input-Lock nutzen, um ungewollte Eingaben zu blockieren.

Damit lässt sich das System flexibel absichern – von offenem Betrieb im abgeschlossenen Netzwerk bis hin zu passwort- und sperrgeschütztem Betrieb in sensiblen Umgebungen.